计算几何：三维竞赛板子

默认 C++17。代码以赛场可复制为目标：变量短、向量公式统一、常用操作 `O(1)`；坐标、交点、距离使用 `ld`，需要完全精确的整数共线/共面判定使用专门的 `I3 + i128` 模块。

0. 约定与算法选择

问题	接口 / 算法	复杂度
点线面关系、交点、距离	向量点积/叉积	`O(1)`
线段间最近点	参数裁剪	`O(1)`
射线与三角形	Moller-Trumbore	`O(1)`
三角形间相交/距离	点面与 9 对边距离	`O(1)`
球、平面、直线相交	投影 / 二次方程	`O(1)`
最小覆盖球	随机增量	期望 `O(n)`，最坏 `O(n^4)`
三维凸包	随机增量 + 地平线	通常 `O(n^2)`
三维最近点对 / 最近邻	KD 树	平均 `O(n log n)`，最坏 `O(n^2)`
三角网格体积、重心	有向四面体求和	`O(F)`
射线与三角网格首交点	逐面 Moller-Trumbore	`O(F)`
射线 / 线段与 AABB	slab 法	`O(1)`
经纬度、大圆距离	单位球向量	`O(1)`

数值规则：

- `sgn(x)` 用于浮点符号判断；不要用 `x==0` 判断几何关系。
- `Point3::operator<` 和 `operator==` 是精确比较，仅用于容器与原始输入去重；浮点点近似相等用 `same(a,b)`。
- 直线 `L3{a,b}` 用两点表示，要求 `a!=b`。函数名决定它被视为无限直线、射线还是闭线段。
- 平面 `Plane{p,n}` 用面上一点和非零法向量表示。三点建面使用 `makePlane(a,b,c)`，要求三点不共线。
- 平面法向量、多面体三角面的方向会影响“正侧/负侧”和有向体积；相关章节会明确方向。
- `EPS` 应按坐标尺度调整。默认适合中等坐标；坐标约 `1e9` 时可改为 `1e-10` 并尽量使用整数判定。

---

1. 基础层：点、向量、直线、平面

这一节是后续模块的公共依赖，建议整体复制。

- `dot(a,b)`：点积；`cross(a,b)`：向量叉积，返回向量。
- `triple(a,b,c)`：标量三重积 `a dot (b cross c)`，也是有向平行六面体体积。
- `norm2(a)` / `len(a)`：长度平方 / 长度。
- `dis2(a,b)` / `dis(a,b)`：两点距离平方 / 距离。
- `unit(a)`：单位向量，要求 `a` 非零。
- `makePlane(a,b,c)`：三点确定平面，法向量为 `(b-a) cross (c-a)`。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ld = long double;
using i128 = __int128_t;

const ld PI = acosl(-1.0L);
const ld EPS = 1e-12L;
const ld INF = numeric_limits<ld>::infinity();
#define cc(x) cout << fixed << setprecision(x);

int sgn(ld x) { return (x>EPS)-(x<-EPS); }
bool eq(ld a, ld b) { return fabsl(a-b)<=EPS; }
ld clamp1(ld x) { return max(-1.0L,min(1.0L,x)); }
ld clamp01(ld x) { return max(0.0L,min(1.0L,x)); }

struct Point3 {
    ld x,y,z;
    Point3(ld x=0,ld y=0,ld z=0):x(x),y(y),z(z){}
    Point3& operator+=(const Point3& p){ x+=p.x,y+=p.y,z+=p.z; return *this; }
    Point3& operator-=(const Point3& p){ x-=p.x,y-=p.y,z-=p.z; return *this; }
    Point3& operator*=(ld k){ x*=k,y*=k,z*=k; return *this; }
    Point3& operator/=(ld k){ x/=k,y/=k,z/=k; return *this; }
    Point3 operator-() const { return {-x,-y,-z}; }
    friend Point3 operator+(Point3 a,const Point3& b){ return a+=b; }
    friend Point3 operator-(Point3 a,const Point3& b){ return a-=b; }
    friend Point3 operator*(Point3 a,ld k){ return a*=k; }
    friend Point3 operator*(ld k,Point3 a){ return a*=k; }
    friend Point3 operator/(Point3 a,ld k){ return a/=k; }
    friend bool operator<(const Point3& a,const Point3& b){
        if(a.x!=b.x) return a.x<b.x;
        return a.y!=b.y?a.y<b.y:a.z<b.z;
    }
    friend bool operator==(const Point3& a,const Point3& b){ return
a.x==b.x&&a.y==b.y&&a.z==b.z; }
    friend bool operator!=(const Point3& a,const Point3& b){ return !(a==b); }
    friend istream& operator>>(istream& in,Point3& p){ return in>>p.x>>p.y>>p.z; }
    friend ostream& operator<<(ostream& out,const Point3& p){ return out<<p.x<<'
'<<p.y<<' '<<p.z; }
};
```

```
#define P3 Point3

struct Line3 {
    P3 a,b;
    Line3(P3 a={},P3 b={}):a(a),b(b){}
};
#define L3 Line3

struct Plane {
    P3 p,n;
    Plane(P3 p={},P3 n={}):p(p),n(n){}
};
struct Sphere {
    P3 o;
    ld r;
    Sphere(P3 o={},ld r=0):o(o),r(r){}
};
struct Circle3 {
    P3 o,n;
    ld r;
};
struct Tri3 { P3 a,b,c; };

ld dot(P3 a,P3 b){ return a.x*b.x+a.y*b.y+a.z*b.z; }
P3 cross(P3 a,P3 b){ return {a.y*b.z-a.z*b.y,a.z*b.x-a.x*b.z,a.x*b.y-a.y*b.x}; }
ld triple(P3 a,P3 b,P3 c){ return dot(a,cross(b,c)); }
ld norm2(P3 a){ return dot(a,a); }
ld len(P3 a){ return sqrtl(max(0.0L,norm2(a))); }
ld dis2(P3 a,P3 b){ return norm2(a-b); }
ld dis(P3 a,P3 b){ return len(a-b); }
ld coord(P3 p,int k){ return k==0?p.x:(k==1?p.y:p.z); } // k=0/1/2: x/y/z
P3 unit(P3 a){ return a/len(a); } // a: 非零向量
bool same(P3 a,P3 b){ return len(a-b)<=EPS; }
Plane makePlane(P3 a,P3 b,P3 c){ return {a,cross(b-a,c-a)}; } // a,b,c: 不共线
ld side(P3 p,Plane s){ return dot(s.n,p-s.p); } // >0 在法向量所指一侧
```

1.1 整数精确共线、共面与定向

普通 `P3` 使用浮点，适合距离和交点。输入为整数且只判断符号时使用本模块：

- `cross128(a,b)`：整数向量叉积。
- `collinear(a,b,c)`：三点是否共线。
- `orient(a,b,c,d)`：标量三重积 `(b-a) dot ((c-a) cross (d-a))`，正负表示 `d` 在有向平面 `abc` 的两侧，零表示四点共面。

坐标绝对值约 `1e9` 时三重积约可到 `1e27`，能装入 `i128`；再大需自行确认范围。

```
struct I3 {
    ll x,y,z;
    friend I3 operator-(I3 a,I3 b){ return {a.x-b.x,a.y-b.y,a.z-b.z}; }
};
```

```
struct V128 { i128 x,y,z; };
V128 cross128(I3 a,I3 b){
    return {(i128)a.y*b.z-(i128)a.z*b.y,(i128)a.z*b.x-(i128)a.x*b.z,(i128)a.x*b.y-
(i128)a.y*b.x};
}
i128 triple128(I3 a,I3 b,I3 c){
    V128 v=cross128(b,c);
    return (i128)a.x*v.x+(i128)a.y*v.y+(i128)a.z*v.z;
}
bool collinear(I3 a,I3 b,I3 c){
    V128 v=cross128(b-a,c-a);
    return v.x==0&&v.y==0&&v.z==0;
}
i128 orient(I3 a,I3 b,I3 c,I3 d){ return triple128(b-a,c-a,d-a); }
```

---

2. 点、直线与线段

2.1 位置关系

- `onLine(p,l)`：点是否在无限直线上。
- `onRay(p,r)`：点是否在从 `r.a` 沿 `r.b-r.a` 出发的射线上。
- `onSeg(p,s)`：点是否在闭线段 `[s.a,s.b]` 上。
- `lineParallel` / `linePerpendicular`：两无限直线方向是否平行 / 垂直，不要求共面。
- `sameLine`：两直线是否重合。

均为 `O(1)`，所有直线要求非退化。

```
bool onLine(P3 p,L3 l){ return len(cross(p-l.a,l.b-l.a))<=EPS; }
bool onRay(P3 p,L3 r){ return onLine(p,r)&&sgn(dot(p-r.a,r.b-r.a))>=0; }
bool onSeg(P3 p,L3 s){ return onLine(p,s)&&sgn(dot(p-s.a,p-s.b))<=0; }
bool lineParallel(L3 a,L3 b){ return len(cross(a.b-a.a,b.b-b.a))<=EPS; }
bool linePerpendicular(L3 a,L3 b){ return sgn(dot(a.b-a.a,b.b-b.a))==0; }
bool sameLine(L3 a,L3 b){ return lineParallel(a,b)&&onLine(b.a,a); }
bool coplanar(L3 a,L3 b){ return sgn(triple(a.b-a.a,b.b-b.a,b.a-a.a))==0; }
```

2.2 投影、对称与点线段距离

- `proj(p,l)`：点到无限直线的投影。
- `refl(p,l)`：点关于无限直线的轴对称点（空间中即绕该轴旋转 `PI`）。
- `nearSeg(p,s)`：闭线段上离 `p` 最近的点。
- `disLine` / `disSeg`：点到无限直线 / 闭线段距离。

```
P3 proj(P3 p,L3 l){ // p: 点; l: 非退化无限直线
    P3 v=l.b-l.a;
    return l.a+v*(dot(p-l.a,v)/norm2(v));
}
P3 refl(P3 p,L3 l){ return proj(p,l)*2-p; }
P3 nearSeg(P3 p,L3 s){ // p: 点; s: 非退化闭线段
    P3 v=s.b-s.a;
    return s.a+v*clamp01(dot(p-s.a,v)/norm2(v));
}
ld disLine(P3 p,L3 l){ return len(cross(p-l.a,l.b-l.a))/len(l.b-l.a); }
ld disSeg(P3 p,L3 s){ return dis(p,nearSeg(p,s)); }
```

2.3 两无限直线最近点、距离与交点

空间两直线可能相交、平行、重合或异面。`lineClosest(a,b)` 对任意两条非退化直线返回 `{距离,a 上最近点,b 上最近点}`；平行时最近点对不唯一，函数固定取 `a.a` 及它在 `b` 上的投影。

`lineCross(a,b,p)` 只在存在唯一交点时返回 `true` 并写入 `p`；平行、重合、异面都返回 `false`。

```
tuple<ld,P3,P3> lineClosest(L3 a,L3 b){ // a,b: 两条非退化无限直线
    P3 u=a.b-a.a,v=b.b-b.a,w=a.a-b.a;
    ld A=dot(u,u),B=dot(u,v),C=dot(v,v),D=dot(u,w),E=dot(v,w);
    ld z=A*C-B*B;
    if(sgn(z)==0){ P3 p=a.a,q=proj(p,b); return {dis(p,q),p,q}; }
    ld s=(B*E-C*D)/z,t=(A*E-B*D)/z;
    P3 p=a.a+u*s,q=b.a+v*t;
    return {dis(p,q),p,q};
}
bool lineCross(L3 a,L3 b,P3& p){ // p: 输出唯一交点
    if(lineParallel(a,b)) return false;
    auto [d,x,y]=lineClosest(a,b);
    if(sgn(d)!=0) return false;
    p=(x+y)/2;
    return true;
}
```

2.4 两闭线段最近点、距离与相交

`segClosest(a,b)` 同时处理点状退化线段，返回 `{距离,a 上最近点,b 上最近点}`。`segCross` 把端点接触和共线重叠都视为相交。复杂度 `O(1)`。

```
tuple<ld,P3,P3> segClosest(L3 a,L3 b){ // a,b: 闭线段，可退化为点
    P3 u=a.b-a.a,v=b.b-b.a,w=a.a-b.a;
    ld A=dot(u,u),B=dot(u,v),C=dot(v,v),D=dot(u,w),E=dot(v,w),s=0,t=0;
    if(sgn(A)==0&&sgn(C)==0) return {dis(a.a,b.a),a.a,b.a};
    if(sgn(A)==0) t=clamp01(E/C);
    else if(sgn(C)==0) s=clamp01(-D/A);
    else{
        ld z=A*C-B*B;
        if(sgn(z)!=0) s=clamp01((B*E-C*D)/z);
```

```
        t=(B*s+E)/C;
        if(t<0) t=0,s=clamp01(-D/A);
        else if(t>1) t=1,s=clamp01((B-D)/A);
    }
    P3 p=a.a+u*s,q=b.a+v*t;
    return {dis(p,q),p,q};
}
bool segCross(L3 a,L3 b){ return sgn(get<0>(segClosest(a,b)))==0; } // 含端点/重叠
```

3. 平面

3.1 点面关系、投影、对称与距离

有向距离沿法向量方向为正。`proj(p,s)` 和 `refl(p,s)` 分别返回点到平面的投影、关于平面的镜像。

```
bool onPlane(P3 p,Plane s){ return sgn(side(p,s))==0; }
ld signedDis(P3 p,Plane s){ return side(p,s)/len(s.n); }
ld disPlane(P3 p,Plane s){ return fabsl(signedDis(p,s)); }
P3 proj(P3 p,Plane s){ return p-s.n*(side(p,s)/norm2(s.n)); }
P3 refl(P3 p,Plane s){ return proj(p,s)*2-p; }
bool planeParallel(Plane a,Plane b){ return len(cross(a.n,b.n))<=EPS; }
bool planePerpendicular(Plane a,Plane b){ return sgn(dot(a.n,b.n))==0; }
bool samePlane(Plane a,Plane b){ return planeParallel(a,b)&&onPlane(b.p,a); }
ld disPlanePlane(Plane a,Plane b){ return planeParallel(a,b)?disPlane(b.p,a):0; }
```

3.2 直线与平面

`linePlaneRel(l,s)` 返回：`0` 平行且不相交，`1` 唯一交点，`2` 直线包含于平面。`linePlaneCross` 仅在唯一交点时返回 `true`。

```
int linePlaneRel(L3 l,Plane s){ // 0 平行,1 唯一交点,2 线在面内
    ld d=dot(s.n,l.b-l.a);
    if(sgn(d)!=0) return 1;
    return onPlane(l.a,s)?2:0;
}
bool linePlaneCross(L3 l,Plane s,P3& p){ // p: 输出唯一交点
    P3 v=l.b-l.a; ld d=dot(s.n,v);
    if(sgn(d)==0) return false;
    p=l.a+v*(dot(s.n,s.p-l.a)/d);
    return true;
}
bool rayPlaneCross(L3 r,Plane s,P3& p,ld& t){ // r: 射线; t,p: 输出参数与交点
    P3 v=r.b-r.a; ld d=dot(s.n,v);
    if(sgn(d)==0) return false;
    t=dot(s.n,s.p-r.a)/d;
    if(t<-EPS) return false;
    p=r.a+v*t;
    return true;
}
```

```
bool segPlaneCross(L3 l,Plane s,P3& p){ // l: 闭线段; p: 输出唯一交点
    ld t;
    return rayPlaneCross(l,s,p,t)&&t<=1+EPS;
}
```

3.3 两平面交线与三平面交点

`planeCross(a,b,l)` 在两平面有唯一交线时返回 `true`。`threePlane(a,b,c,p)` 在三个平面有唯一公共点时返回 `true`。都直接使用对称向量公式，不依赖选择某个坐标平面。

```
bool planeCross(Plane a,Plane b,L3& l){ // l: 输出交线
    P3 v=cross(a.n,b.n);
    ld z=norm2(v);
    if(sgn(z)==0) return false;
    ld x=dot(a.n,a.p),y=dot(b.n,b.p);
    P3 p=cross(b.n*x-a.n*y,v)/z;
    l={p,p+v};
    return true;
}
bool threePlane(Plane a,Plane b,Plane c,P3& p){ // p: 输出唯一公共点
    ld z=triple(a.n,b.n,c.n);
    if(sgn(z)==0) return false;
    ld x=dot(a.n,a.p),y=dot(b.n,b.p),w=dot(c.n,c.p);
    p=(cross(b.n,c.n)*x+cross(c.n,a.n)*y+cross(a.n,b.n)*w)/z;
    return true;
}
```

3.4 平面一般式 Ax+By+Cz+D=0

`ABCD(s)` 返回平面系数。`fromABCD` 从一般式建面，并选择绝对值最大的非零系数求面上一点，数值更稳定；要求 `A,B,C` 不全为零。

```
array<ld,4> ABCD(Plane s){ return {s.n.x,s.n.y,s.n.z,-dot(s.n,s.p)}; }
Plane fromABCD(ld A,ld B,ld C,ld D){ // Ax+By+Cz+D=0
    P3 p;
    if(fabsl(A)>=fabsl(B)&&fabsl(A)>=fabsl(C)) p={-D/A,0,0};
    else if(fabsl(B)>=fabsl(C)) p={0,-D/B,0};
    else p={0,0,-D/C};
    return {p,{A,B,C}};
}
```

3.5 角度

以下均返回较小的无向夹角：直线夹角、平面二面角在 `[0,PI/2]`；线面角在 `[0,PI/2]`。需要有向角时见下一节。

```
ld angleVec(P3 a,P3 b){ return acosl(clamp1(dot(a,b)/len(a)/len(b))); } // [0,PI]
ld angleLine(L3 a,L3 b){ return acosl(clamp1(fabsl(dot(a.b-a.a,b.b-b.a))/len(a.b-
a.a)/len(b.b-b.a))); }
ld anglePlane(Plane a,Plane b){ return
acosl(clamp1(fabsl(dot(a.n,b.n))/len(a.n)/len(b.n))); }
ld angleLinePlane(L3 l,Plane s){ return asinl(clamp1(fabsl(dot(l.b-
l.a,s.n))/len(l.b-l.a)/len(s.n))); }
```

---

4. 旋转与平面局部坐标

4.1 Rodrigues 绕轴旋转

`rotAxis(v,k,a)` 把向量 `v` 绕过原点的单位轴方向 `k` 按右手定则旋转 `a` 弧度。`rotLine(p,l,a)` 把点 `p` 绕空间有向直线 `l.a->l.b` 旋转。

`signedAngle(a,b,n)` 返回从 `a` 到 `b` 绕单位法向量 `n` 的有向角，范围 `[-PI,PI]`；要求 `a,b` 都垂直于 `n`。

```
P3 rotAxis(P3 v,P3 k,ld a){ // k: 单位轴; a: 右手定则弧度
    ld c=cosl(a),s=sinl(a);
    return v*c+cross(k,v)*s+k*dot(k,v)*(1-c);
}
P3 rotLine(P3 p,L3 l,ld a){ return l.a+rotAxis(p-l.a,unit(l.b-l.a),a); }
ld signedAngle(P3 a,P3 b,P3 n){ return atan2l(dot(n,cross(a,b)),dot(a,b)); }
```

4.2 平面正交基与二维投影

`makeBasis(s)` 为平面建立右手正交坐标系 `{o,x,y,n}`，满足 `x cross y = n`。`to2(p,b)` 把空间点投影为局部二维坐标；`to3(q,b)` 把二维坐标还原到平面。

这是处理空间共面多边形、将三维问题降成二维问题的标准接口，均为 `O(1)`。

```
struct Point2 { ld x,y; };
struct Basis3 { P3 o,x,y,n; };
Basis3 makeBasis(Plane s){ // s: 非退化平面
    P3 n=unit(s.n);
    P3 t=fabsl(n.x)<0.9L?P3{1,0,0}:P3{0,1,0};
    P3 x=unit(cross(t,n)),y=cross(n,x);
    return {s.p,x,y,n};
}
Point2 to2(P3 p,Basis3 b){ P3 v=p-b.o; return {dot(v,b.x),dot(v,b.y)}; }
P3 to3(Point2 p,Basis3 b){ return b.o+b.x*p.x+b.y*p.y; }
```

---

5. 空间三角形与共面多边形

5.1 三角形面积、法向量与重心坐标

- `triArea(a,b,c)`：三角形面积。
- `barycentric(p,t)`：返回重心坐标 `{u,v,w}`，满足 `p=u*a+v*b+w*c`、`u+v+w=1`；它按 `p` 在三角形平面的正交投影计算，判断包含前仍要检查共面。
- `inTri(p,t)`：返回 `0` 不在三角形上、`1` 边界、`2` 相对内部。

```
P3 triNormal(Tri3 t){ return cross(t.b-t.a,t.c-t.a); }
ld triArea(Tri3 t){ return len(triNormal(t))/2; }
array<ld,3> barycentric(P3 p,Tri3 t){ // t: 非退化三角形
    P3 u=t.b-t.a,v=t.c-t.a,w=p-t.a;
    ld a=dot(u,u),b=dot(u,v),c=dot(v,v),d=dot(w,u),e=dot(w,v);
    ld z=a*c-b*b;
    ld y=(a*e-b*d)/z,x=(c*d-b*e)/z;
    return {1-x-y,x,y};
}
int inTri(P3 p,Tri3 t){ // 0 外,1 边界,2 内部
    Plane s{t.a,triNormal(t)};
    if(!onPlane(p,s)) return 0;
    auto q=barycentric(p,t);
    if(q[0]<-EPS||q[1]<-EPS||q[2]<-EPS) return 0;
    return sgn(q[0])==0||sgn(q[1])==0||sgn(q[2])==0?1:2;
}
```

5.2 点到三角形最近点

`nearTri(p,t)` 返回闭三角形上离点 `p` 最近的点，内部、边和顶点区域均用点积判定，不需要先投影。`disTri` 返回距离。复杂度 `O(1)`。

```
P3 nearTri(P3 p,Tri3 t){ // p: 点; t: 非退化闭三角形
    P3 a=t.a,b=t.b,c=t.c,ab=b-a,ac=c-a,ap=p-a;
    ld d1=dot(ab,ap),d2=dot(ac,ap);
    if(d1<=0&&d2<=0) return a;
    P3 bp=p-b; ld d3=dot(ab,bp),d4=dot(ac,bp);
    if(d3>=0&&d4<=d3) return b;
    ld vc=d1*d4-d3*d2;
    if(vc<=0&&d1>=0&&d3<=0) return a+ab*(d1/(d1-d3));
    P3 cp=p-c; ld d5=dot(ab,cp),d6=dot(ac,cp);
    if(d6>=0&&d5<=d6) return c;
    ld vb=d5*d2-d1*d6;
    if(vb<=0&&d2>=0&&d6<=0) return a+ac*(d2/(d2-d6));
    ld va=d3*d6-d5*d4;
    if(va<=0&&d4-d3>=0&&d5-d6>=0){
        ld w=(d4-d3)/((d4-d3)+(d5-d6));
        return b+(c-b)*w;
    }
    ld z=1/(va+vb+vc),v=vb*z,w=vc*z;
    return a+ab*v+ac*w;
}
```

```
ld disTri(P3 p,Tri3 t){ return dis(p,nearTri(p,t)); }
```

5.3 射线 / 线段与三角形

`rayTri(r,t,k,p,cull)` 使用 Moller-Trumbore：射线从 `r.a` 出发，方向为 `r.b-r.a`；相交时返回 `true`，`k` 是参数且 `p=r.a+k*(r.b-r.a)`。

- `cull=false`：三角形双面可见，边界接触算相交。
- `cull=true`：只接受法向量朝向射线的正面。
- 射线与三角形共面时没有唯一穿透点，本函数返回 `false`。

`segTri(s,t,p)` 判断闭线段与三角形是否有唯一的非共面交点。共面相交由下一节 `triCross` 完整处理。

```
bool rayTri(L3 r,Tri3 t,ld& k,P3& p,bool cull=false){ // k,p: 输出参数与交点
    P3 d=r.b-r.a,e1=t.b-t.a,e2=t.c-t.a,h=cross(d,e2);
    ld z=dot(e1,h);
    if(cull){ if(z<=EPS) return false; }
    else if(fabsl(z)<=EPS) return false;
    ld f=1/z; P3 s=r.a-t.a;
    ld u=f*dot(s,h);
    if(u<-EPS||u>1+EPS) return false;
    P3 q=cross(s,e1); ld v=f*dot(d,q);
    if(v<-EPS||u+v>1+EPS) return false;
    k=f*dot(e2,q);
    if(k<-EPS) return false;
    p=r.a+d*k;
    return true;
}
bool segTri(L3 s,Tri3 t,P3& p){ // s: 闭线段; p: 输出非共面交点
    if(same(s.a,s.b)) return inTri(s.a,t)>0?(p=s.a,true):false;
    ld k;
    return rayTri(s,t,k,p)&&k<=1+EPS;
}
```

5.4 两空间三角形相交与距离

`triDistance(a,b)` 返回两个闭三角形的最短距离。它先检测每条边是否穿过另一三角形，再取 6 个点到三角形距离和 9 对边间距离的最小值；因此异面穿透、共面交叉、顶点/边接触都能处理。`triCross` 在距离为零时返回 `true`。复杂度是常数。

```
ld triDistance(Tri3 a,Tri3 b){ // a,b: 非退化闭三角形
    array<P3,3> x={a.a,a.b,a.c},y={b.a,b.b,b.c};
    P3 p;
    for(int i=0;i<3;i++){
        if(segTri({x[i],x[(i+1)%3]},b,p)) return 0;
        if(segTri({y[i],y[(i+1)%3]},a,p)) return 0;
    }
    ld ans=INF;
```

```
    for(P3 q:x) ans=min(ans,disTri(q,b));
    for(P3 q:y) ans=min(ans,disTri(q,a));
    for(int i=0;i<3;i++) for(int j=0;j<3;j++)
        ans=min(ans,get<0>(segClosest({x[i],x[(i+1)%3]},{y[j],y[(j+1)%3]})));
    return ans;
}
bool triCross(Tri3 a,Tri3 b){ return sgn(triDistance(a,b))==0; }
```

线段到三角形距离

`segTriDistance(s,t)` 返回闭线段与闭三角形的最短距离。若线段穿过或接触三角形返回 `0`；否则比较两个端点到三角形和线段到三条边的距离。复杂度 `O(1)`，退化为点的线段也能处理。

```
ld segTriDistance(L3 s,Tri3 t){ // s: 闭线段; t: 非退化闭三角形
    P3 p;
    if(segTri(s,t,p)) return 0;
    ld ans=min(disTri(s.a,t),disTri(s.b,t));
    array<P3,3> a={t.a,t.b,t.c};
    for(int i=0;i<3;i++) ans=min(ans,get<0>(segClosest(s,{a[i],a[(i+1)%3]})));
    return ans;
}
```

5.5 共面简单多边形

顶点按边界顺序给出，不重复首点。

- `polyAreaVec(p)`：面积向量，方向由右手定则决定，长度等于面积。
- `polyArea(p)`：普通面积。
- `polyCentroid(p)`：均匀薄片的面积重心，支持凹简单多边形，要求面积非零。
- `inPlanarPoly(q,p)`：返回 `0` 外、`1` 边界、`2` 内；若 `q` 不在多边形平面上直接返回 `0`。

均为 `O(n)`。

```
P3 polyAreaVec(const vector<P3>& p){
    P3 s{}; int n=p.size();
    for(int i=0;i<n;i++) s+=cross(p[i],p[(i+1)%n]);
    return s/2;
}
ld polyArea(const vector<P3>& p){ return len(polyAreaVec(p)); }
P3 polyCentroid(const vector<P3>& p){ // p: 面积非零的共面简单多边形
    P3 n=unit(polyAreaVec(p)),g{}; ld s=0;
    for(int i=1;i+1<(int)p.size();i++){
        ld w=dot(cross(p[i]-p[0],p[i+1]-p[0]),n)/2;
        g+=(p[0]+p[i]+p[i+1])*(w/3),s+=w;
    }
    return g/s;
}
ld cross2(Point2 a,Point2 b,Point2 c){ return (b.x-a.x)*(c.y-a.y)-(b.y-a.y)*(c.x-
a.x); }
bool onSeg2(Point2 p,Point2 a,Point2 b){
```

```
    return sgn(cross2(a,b,p))==0&&sgn((p.x-a.x)*(p.x-b.x)+(p.y-a.y)*(p.y-b.y))<=0;
}
int inPlanarPoly(P3 q,const vector<P3>& p){ // q: 查询点; p: 共面简单多边形
    if(p.size()<3) return 0;
    P3 n=polyAreaVec(p);
    Plane s{p[0],n};
    if(!onPlane(q,s)) return 0;
    Basis3 bs=makeBasis(s); Point2 x=to2(q,bs);
    vector<Point2> a; for(P3 v:p) a.push_back(to2(v,bs));
    bool in=false; int m=a.size();
    for(int i=0;i<m;i++){
        Point2 u=a[i],v=a[(i+1)%m];
        if(onSeg2(x,u,v)) return 1;
        bool f=(u.y>x.y)!=(v.y>x.y);
        if(f&&sgn(cross2(u,v,x))==(v.y>u.y?1:-1)) in=!in;
    }
    return in?2:0;
}
```

5.6 平面与三角形交集、半空间裁剪

`planeTriCross(s,t)` 返回平面与闭三角形的交集：

- 空数组：不相交；
- 1 个点：只接触一个顶点；
- 2 个点：交集是一条线段；
- 3 个点：整个三角形与平面共面。

`cutPoly3(p,s,pos)` 用平面裁剪共面简单多边形：`pos=true` 保留 `side>=0` 的闭半空间，`false` 保留 `side<=0`。输入多边形本身可以位于任意平面，复杂度 `O(n)`。

```
vector<P3> planeTriCross(Plane s,Tri3 t){ // s: 平面; t: 闭三角形
    array<P3,3> a={t.a,t.b,t.c};
    array<ld,3> d={side(a[0],s),side(a[1],s),side(a[2],s)};
    if(sgn(d[0])==0&&sgn(d[1])==0&&sgn(d[2])==0) return {a[0],a[1],a[2]};
    vector<P3> q;
    auto add=[&](P3 p){ for(P3 x:q) if(same(x,p)) return; q.push_back(p); };
    for(int i=0;i<3;i++){
        int j=(i+1)%3,di=sgn(d[i]),dj=sgn(d[j]);
        if(di==0) add(a[i]);
        if(di*dj<0) add(a[i]+(a[j]-a[i])*(d[i]/(d[i]-d[j])));
    }
    return q;
}
vector<P3> cutPoly3(const vector<P3>& p,Plane s,bool pos=true){ // pos=1 保留法向正侧
    vector<P3> q; int n=p.size(); ld w=pos?1:-1;
    for(int i=0;i<n;i++){
        P3 a=p[i],b=p[(i+1)%n]; ld x=side(a,s)*w,y=side(b,s)*w;
        if(x>=-EPS) q.push_back(a);
        if(sgn(x)*sgn(y)<0) q.push_back(a+(b-a)*(x/(x-y)));
```

```
    }
    return q;
}
```

6. 球与空间圆

6.1 基础关系与度量

`sphereRel(p,s)` 返回点相对球面的关系：`-1` 球内、`0` 球面、`1` 球外。球表面积、体积、球冠面积和球冠体积均为 `O(1)`。

```
int sphereRel(P3 p,Sphere s){ return sgn(dis2(p,s.o)-s.r*s.r.r); }
ld sphereArea(ld r){ return 4*PI*r*r; }
ld sphereVolume(ld r){ return 4*PI*r*r*r/3; }
ld capArea(ld r,ld h){ return 2*PI*r*h; } // 球半径 r，球冠高 h
ld capVolume(ld r,ld h){ return PI*h*h*(3*r-h)/3; }
```

6.2 直线 / 线段与球面交点

`lineSphere(l,s)` 返回无限直线与球面的 0、1、2 个交点。`segSphere` 只保留闭线段上的点。结果沿直线参数从小到大排列。

```
vector<P3> lineSphere(L3 l,Sphere s){ // l: 非退化无限直线; s: 球
    P3 d=l.b-l.a,m=l.a-s.o;
    ld A=dot(d,d),B=2*dot(m,d),C=dot(m,m)-s.r*s.r,z=B*B-4*A*C;
    if(sgn(z)<0) return {};
    if(sgn(z)==0) return {l.a+d*(-B/(2*A))};
    z=sqrtl(max(0.0L,z));
    return {l.a+d*((-B-z)/(2*A)),l.a+d*((-B+z)/(2*A))};
}
vector<P3> segSphere(L3 l,Sphere s){
    vector<P3> q;
    for(P3 p:lineSphere(l,s)) if(onSeg(p,l)) q.push_back(p);
    return q;
}
```

6.3 平面与球的交圆

`planeSphere(s,b,c)` 在平面 `s` 与球 `b` 相交或相切时返回 `true`，并输出交圆 `c`。相切时半径为零；相离返回 `false`。

```
bool planeSphere(Plane s,Sphere b,Circle3& c){ // c: 输出交圆
    ld d=disPlane(b.o,s);
    if(sgn(d-b.r)>0) return false;
    c={proj(b.o,s),unit(s.n),sqrtl(max(0.0L,b.r*b.r-d*d))};
    return true;
}
```

6.4 两球关系、交圆与交体积

`sphereRelation(a,b)` 返回：`-2` 内含，`-1` 内切，`0` 相交成圆，`1` 外切，`2` 相离，`3` 重合。`sphereCrossCircle` 输出相交圆；重合球交集不是唯一圆，因此返回 `false`。

`sphereOverlap` 返回两个闭球的交体积，包含、相离、相切均已处理。

```
int sphereRelation(Sphere a,Sphere b){
    ld d=dis(a.o,b.o),sm=a.r+b.r,df=fabsl(a.r-b.r);
    if(sgn(d)==0&&sgn(a.r-b.r)==0) return 3;
    if(sgn(d-sm)>0) return 2;
    if(sgn(d-sm)==0) return 1;
    if(sgn(d-df)>0) return 0;
    if(sgn(d-df)==0) return -1;
    return -2;
}
bool sphereCrossCircle(Sphere a,Sphere b,Circle3& c){ // c: 输出相交圆
    P3 v=b.o-a.o; ld d=len(v);
    if(sgn(d)==0||sgn(d-a.r-b.r)>0||sgn(d-fabsl(a.r-b.r))<0) return false;
    ld x=(a.r*a.r-b.r*b.r+d*d)/(2*d);
    c={a.o+v*(x/d),unit(v),sqrtl(max(0.0L,a.r*a.r-x*x))};
    return true;
}
ld sphereOverlap(Sphere a,Sphere b){ // 两闭球交体积
    ld d=dis(a.o,b.o);
    if(sgn(d-a.r-b.r)>=0) return 0;
    if(sgn(d-fabsl(a.r-b.r))<=0) return sphereVolume(min(a.r,b.r));
    ld x=a.r+b.r-d;
    return PI*x*x*(d*d+2*d*(a.r+b.r)-3*(a.r-b.r)*(a.r-b.r))/(12*d);
}
```

6.5 三点外接圆与四点外接球

`circumCircle3(a,b,c)` 返回不共线三点所在平面内的外接圆。`circumSphere4(a,b,c,d,s)` 在四点不共面时返回 `true` 并输出唯一外接球。

```
Circle3 circumCircle3(P3 a,P3 b,P3 c){ // a,b,c: 不共线
    P3 u=b-a,v=c-a,n=cross(u,v);
    P3 o=a+(cross(v,n)*norm2(u)+cross(n,u)*norm2(v))/(2*norm2(n));
    return {o,unit(n),dis(o,a)};
}
bool circumSphere4(P3 a,P3 b,P3 c,P3 d,Sphere& s){ // s: 输出外接球
    P3 u=b-a,v=c-a,w=d-a;
    ld z=2*triple(u,v,w);
    if(sgn(z)==0) return false;
    P3 o=a+(cross(v,w)*norm2(u)+cross(w,u)*norm2(v)+cross(u,v)*norm2(w))/z;
    s={o,dis(o,a)};
    return true;
}
```

6.6 空间圆查询

空间圆 `Circle3{o,n,r}` 位于过 `o`、法向量为 `n` 的平面内。

- `circlePointRange(p,c)` 返回 `{最近距离,最近点,最远距离,最远点}`。若 `p` 在圆轴上，最近/最远点都不唯一，函数固定选择局部基的 x 方向及反方向。
- `lineCircle(l,c)` 返回无限直线与圆周的 0、1、2 个交点，同时处理直线位于圆平面内和穿过圆平面的情况。
- `circleCross(a,b)` 返回两个空间圆的有限交点。两圆重合时交点无限多，返回空数组。

```
tuple<ld,P3,ld,P3> circlePointRange(P3 p,Circle3 c){ // 点到圆周最近/最远点
    Plane s{c.o,c.n}; P3 h=proj(p,s),v=h-c.o;
    P3 u=sgn(len(v))?unit(v):makeBasis(s).x;
    P3 a=c.o+u*c.r,b=c.o-u*c.r;
    return {dis(p,a),a,dis(p,b),b};
}
vector<P3> lineCircle(L3 l,Circle3 c){ // l: 非退化无限直线
    Plane s{c.o,c.n}; int z=linePlaneRel(l,s);
    if(z==2) return lineSphere(l,{c.o,c.r});
    if(z==0) return {};
    P3 p; linePlaneCross(l,s,p);
    return eq(dis2(p,c.o),c.r*c.r)?vector<P3>{p}:vector<P3>{};
}
vector<P3> circleCross(Circle3 a,Circle3 b){ // 返回有限交点
    Plane x{a.o,a.n},y{b.o,b.n}; L3 l;
    if(planeCross(x,y,l)){
        vector<P3> q;
        for(P3 p:lineSphere(l,{a.o,a.r}))
            if(onPlane(p,y)&&eq(dis2(p,b.o),b.r*b.r)) q.push_back(p);
        return q;
    }
    if(!samePlane(x,y)) return {};
    P3 v=b.o-a.o; ld d=len(v);
    if(sgn(d)==0) return {};
    if(sgn(d-a.r-b.r)>0||sgn(d-fabsl(a.r-b.r))<0) return {};
    ld t=(a.r*a.r-b.r*b.r+d*d)/(2*d),h2=a.r*a.r-t*t;
    P3 p=a.o+v*(t/d);
    if(sgn(h2)==0) return {p};
    P3 q=unit(cross(unit(a.n),v))*sqrtl(max(0.0L,h2));
    return {p-q,p+q};
}
ld circleLength(Circle3 c){ return 2*PI*c.r; }
ld circleArea(Circle3 c){ return PI*c.r*c.r; }
```

6.7 最小覆盖球

`minCoverSphere(p)` 求覆盖全部点的最小闭球。随机增量在固定三维中期望 `O(n)`；四层循环给出最坏 `O(n^4)`。空集返回原点、半径 `0`。

辅助 `ball3`、`ball4` 允许共线、共面和重复点：四点共面时会从两点球、三点球候选中选能覆盖四点且半径最小者。

```
Sphere ball2(P3 a,P3 b){ return {(a+b)/2,dis(a,b)/2}; }
Sphere ball3(P3 a,P3 b,P3 c){ // 覆盖三点的最小球
    if(len(cross(b-a,c-a))>EPS){ auto q=circumCircle3(a,b,c); return {q.o,q.r}; }
    if(dis2(a,b)>=dis2(a,c)&&dis2(a,b)>=dis2(b,c)) return ball2(a,b);
    if(dis2(a,c)>=dis2(b,c)) return ball2(a,c);
    return ball2(b,c);
}
bool outside(P3 p,Sphere s){ return s.r<0||sgn(dis2(p,s.o)-s.r*s.r)>0; }
Sphere ball4(P3 a,P3 b,P3 c,P3 d){ // 覆盖四点的最小球
    array<P3,4> p={a,b,c,d}; vector<Sphere> q;
    for(int i=0;i<4;i++) for(int j=i+1;j<4;j++) q.push_back(ball2(p[i],p[j]));
    for(int i=0;i<4;i++) for(int j=i+1;j<4;j++) for(int k=j+1;k<4;k++)
q.push_back(ball3(p[i],p[j],p[k]));
    Sphere s;
    if(circumSphere4(a,b,c,d,s)) q.push_back(s);
    Sphere ans{{},INF};
    for(auto x:q){
        bool ok=true; for(P3 y:p) ok&=!outside(y,x);
        if(ok&&x.r<ans.r) ans=x;
    }
    return ans;
}
Sphere minCoverSphere(vector<P3> p){ // p: 任意点集，会被打乱
    mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
    shuffle(p.begin(),p.end(),rng);
    Sphere s{{},-1};
    for(int i=0;i<(int)p.size();i++) if(outside(p[i],s)){
        s={p[i],0};
        for(int j=0;j<i;j++) if(outside(p[j],s)){
            s=ball2(p[i],p[j]);
            for(int k=0;k<j;k++) if(outside(p[k],s)){
                s=ball3(p[i],p[j],p[k]);
                for(int l=0;l<k;l++) if(outside(p[l],s))
s=ball4(p[i],p[j],p[k],p[l]);
            }
        }
    }
    return s.r<0?Sphere{{},0}:s;
}
```

7. 四面体、三角网格与多面体

7.1 四面体有向体积

`tetraSigned(a,b,c,d)` 返回有向体积，绝对值是普通体积。符号由点序决定；`tetraVolume` 直接返回非负值。

```
ld tetraSigned(P3 a,P3 b,P3 c,P3 d){ return triple(b-a,c-a,d-a)/6; }
ld tetraVolume(P3 a,P3 b,P3 c,P3 d){ return fabsl(tetraSigned(a,b,c,d)); }
P3 tetraCentroid(P3 a,P3 b,P3 c,P3 d){ return (a+b+c+d)/4; }
Sphere tetraInsphere(P3 a,P3 b,P3 c,P3 d){ // 非退化四面体的内切球
    ld A=triArea({b,c,d}),B=triArea({a,c,d}),C=triArea({a,b,d}),D=triArea({a,b,c});
    ld s=A+B+C+D;
    return {(a*A+b*B+c*C+d*D)/s,3*tetraVolume(a,b,c,d)/s};
}
```

7.2 闭合三角网格面积、体积与重心

`vector<Tri3> f` 表示闭合三角网格。每个面的三个顶点从多面体外部看应为逆时针，即 `cross(b-a,c-a)` 指向外侧。

- `meshArea(f)`：表面积。
- `meshSignedVolume(f)`：有向体积；整体面方向正确时为正，全部反向时为负。
- `meshVolume(f)`：普通体积。
- `meshCentroid(f)`：均匀实体的体积重心，支持凹但要求闭合、定向一致、体积非零。

原点无需在多面体内部。原理是把每个面与原点构成有向四面体，复杂度 `O(F)`。

```
ld meshArea(const vector<Tri3>& f){
    ld s=0; for(auto t:f) s+=triArea(t); return s;
}
ld meshSignedVolume(const vector<Tri3>& f){
    ld s=0; for(auto t:f) s+=triple(t.a,t.b,t.c)/6; return s;
}
ld meshVolume(const vector<Tri3>& f){ return fabsl(meshSignedVolume(f)); }
P3 meshCentroid(const vector<Tri3>& f){ // f: 闭合且定向一致，体积非零
    P3 g{}; ld s=0;
    for(auto t:f){
        ld v=triple(t.a,t.b,t.c)/6;
        g+=(t.a+t.b+t.c)*(v/4),s+=v;
    }
    return g/s;
}
```

7.3 点在凸多面体内

`inConvexPoly(p,f)` 返回 `0` 外、`1` 边界、`2` 内部。要求每个三角面的法向量指向外侧；凸性保证点在所有面的非正侧就位于内部。复杂度 `O(F)`。

```
int inConvexPoly(P3 p,const vector<Tri3>& f){ // f: 外向三角面组成的闭凸多面体
    bool bd=false;
    for(auto t:f){
        int s=sgn(dot(triNormal(t),p-t.a));
        if(s>0) return 0;
        if(s==0) bd=true;
    }
    return bd?1:2;
}
```

7.4 点在一般闭合三角网格内：立体角

`inMesh(p,f)` 也返回 `0/1/2`，可处理凹闭合多面体。先检查边界，再累加每个三角面对 `p` 的有向立体角；内部总和绝对值约为 `4PI`，外部约为 `0`。要求网格封闭、无自交、面方向一致，复杂度 `O(F)`。

```
ld solidAngle(P3 p,Tri3 t){ // 三角面 t 对观察点 p 的有向立体角
    P3 a=t.a-p,b=t.b-p,c=t.c-p;
    ld la=len(a),lb=len(b),lc=len(c);
    ld y=la*lb*lc+dot(a,b)*lc+dot(b,c)*la+dot(c,a)*lb;
    return 2*atan2l(triple(a,b,c),y);
}
int inMesh(P3 p,const vector<Tri3>& f){ // f: 闭合、无自交、定向一致
    for(auto t:f) if(inTri(p,t)) return 1;
    ld s=0; for(auto t:f) s+=solidAngle(p,t);
    return fabsl(s)>2*PI?2:0;
}
```

---

8. 三维静态凸包：随机增量

`convexHull3(p)` 返回 `{点数组,三角面下标}`。输出面均朝外，因而可以直接转成 `Tri3` 供上一节求面积、体积、重心和点包含。

算法流程：

1. 随机打乱并找四个不共面的初始点；
2. 对每个新点删除所有可见面；
3. 可见区域中只出现一次的有向边组成地平线；
4. 用地平线反向边和新点建立朝外的新面。

通常复杂度 `O(n^2)`、空间 `O(n)`。如果所有点共面或不足四个非共面点，返回空面集；这种情况应投影到二维后求二维凸包。大量几乎共面的浮点输入需要按题目尺度调整 `EPS`。

```
struct Hull3 {
    vector<P3> p;
    vector<array<int,3>> f;
};
Hull3 convexHull3(vector<P3> p){ // p: 空间点集
    sort(p.begin(),p.end());
```

```
    p.erase(unique(p.begin(),p.end()),p.end());
    mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
    shuffle(p.begin(),p.end(),rng);
    int n=p.size();
    if(n<4) return {p,{}};
    int b=1; while(b<n&&same(p[0],p[b])) b++;
    if(b==n) return {p,{}};
    swap(p[1],p[b]);
    int c=2; while(c<n&&len(cross(p[1]-p[0],p[c]-p[0]))<=EPS) c++;
    if(c==n) return {p,{}};
    swap(p[2],p[c]);
    int d=3; while(d<n&&fabsl(triple(p[1]-p[0],p[2]-p[0],p[d]-p[0]))<=EPS) d++;
    if(d==n) return {p,{}};
    swap(p[3],p[d]);

    vector<array<int,3>> f;
    auto add0=[&](int a,int b,int c,int in){
        array<int,3> q={a,b,c};
        if(triple(p[b]-p[a],p[c]-p[a],p[in]-p[a])>0) swap(q[1],q[2]);
        f.push_back(q);
    };
    add0(0,1,2,3),add0(0,3,1,2),add0(0,2,3,1),add0(1,3,2,0);

    auto key=[](int a,int b){ return (uint64_t)(uint32_t)a<<32|(uint32_t)b; };
    for(int i=4;i<n;i++){
        vector<char> vis(f.size()); bool any=false;
        for(int j=0;j<(int)f.size();j++){
            auto q=f[j];
            vis[j]=sgn(dot(cross(p[q[1]]-p[q[0]],p[q[2]]-p[q[0]]),p[i]-p[q[0]]))>0;
            any|=vis[j];
        }
        if(!any) continue;
        unordered_map<uint64_t,pair<int,int>> e;
        auto edge=[&](int a,int b){
            auto it=e.find(key(b,a));
            if(it!=e.end()) e.erase(it);
            else e[key(a,b)]={a,b};
        };
        vector<array<int,3>> g;
        for(int j=0;j<(int)f.size();j++){
            auto q=f[j];
            if(!vis[j]) g.push_back(q);
            else edge(q[0],q[1]),edge(q[1],q[2]),edge(q[2],q[0]);
        }
        for(auto [z,q]:e) g.push_back({q.first,q.second,i});
        f.swap(g);
    }
    return {move(p),move(f)};
}
vector<Tri3> hullFaces(const Hull3& h){ // 下标面转坐标面
    vector<Tri3> f; f.reserve(h.f.size());
    for(auto q:h.f) f.push_back({h.p[q[0]],h.p[q[1]],h.p[q[2]]});
```

```
    return f;
}
```

8.1 三维凸包常用统计

凸包三角面数在一般位置下满足 `F=2V-4`、边数 `E=3V-6`。有共面点时增量算法仍会把共面多边形面三角化，面数取决于实际参与凸包的顶点和三角化方式。

```
tuple<ld,ld,P3> hullInfo(const Hull3& h){ // 返回 {表面积,体积,体积重心}
    auto f=hullFaces(h);
    return {meshArea(f),meshVolume(f),meshCentroid(f)};
}
```

---

9. 三维 KD 树：最近邻与最近点对

`KD3 p(points)` 建立静态 KD 树，输入点自动附带原下标。

- `near2(q,ban)`：查询点 `q` 到最近输入点的距离平方；编号为 `ban` 的点不参与，传 `-1` 表示不排除。
- `closestPair()`：返回输入点集中最近两点的实际距离；不足两个点返回 `INF`。

建树按当前包围盒最长轴使用 `nth_element`，平均查询 `O(log n)`，最近点对平均 `O(n log n)`；极端构造下最坏 `O(n^2)`。空间 `O(n)`。

```
struct KD3 {
    struct Item { P3 p; int id; };
    struct Node { P3 p,mn,mx; int id=-1,l=-1,r=-1; };
    vector<Item> a;
    vector<Node> t;
    int rt=-1;
    KD3(vector<P3> p={}){
        for(int i=0;i<(int)p.size();i++) a.push_back({p[i],i});
        t.resize(p.size());
        if(!p.empty()) rt=build(0,p.size());
    }
    int build(int l,int r){
        if(l>=r) return -1;
        P3 mn=a[l].p,mx=a[l].p;
        for(int i=l+1;i<r;i++){
            mn.x=min(mn.x,a[i].p.x),mn.y=min(mn.y,a[i].p.y),mn.z=min(mn.z,a[i].p.z);
            mx.x=max(mx.x,a[i].p.x),mx.y=max(mx.y,a[i].p.y),mx.z=max(mx.z,a[i].p.z);
        }
        P3 w=mx-mn; int k=w.y>w.x?1:0; if(coord(w,2)>coord(w,k)) k=2;
        int m=(l+r)/2;
        nth_element(a.begin()+l,a.begin()+m,a.begin()+r,[&](Item x,Item y){ return
coord(x.p,k)<coord(y.p,k); });
        t[m].p=a[m].p,t[m].id=a[m].id;
        t[m].l=build(l,m),t[m].r=build(m+1,r);
        pull(m);
```

```
        return m;
    }
    void pull(int o){
        t[o].mn=t[o].mx=t[o].p;
        for(int v:{t[o].l,t[o].r}) if(v!=-1){

 t[o].mn.x=min(t[o].mn.x,t[v].mn.x),t[o].mn.y=min(t[o].mn.y,t[v].mn.y),t[o].mn.z=min
(t[o].mn.z,t[v].mn.z);

 t[o].mx.x=max(t[o].mx.x,t[v].mx.x),t[o].mx.y=max(t[o].mx.y,t[v].mx.y),t[o].mx.z=max
(t[o].mx.z,t[v].mx.z);
        }
    }
    ld box2(P3 p,int o) const{
        ld s=0;
        for(int k=0;k<3;k++){
            ld x=coord(p,k),lo=coord(t[o].mn,k),hi=coord(t[o].mx,k);
            if(x<lo) s+=(lo-x)*(lo-x); else if(x>hi) s+=(x-hi)*(x-hi);
        }
        return s;
    }
    void ask(int o,P3 p,int ban,ld& ans) const{
        if(o==-1) return;
        if(t[o].id!=ban) ans=min(ans,dis2(p,t[o].p));
        int x=t[o].l,y=t[o].r;
        ld dx=x==-1?INF:box2(p,x),dy=y==-1?INF:box2(p,y);
        if(dx>dy) swap(x,y),swap(dx,dy);
        if(dx<ans) ask(x,p,ban,ans);
        if(dy<ans) ask(y,p,ban,ans);
    }
    ld near2(P3 p,int ban=-1) const{ ld ans=INF; ask(rt,p,ban,ans); return ans; }
    ld closestPair() const{
        if(a.size()<2) return INF;
        ld ans=INF;
        for(auto q:a) ans=min(ans,near2(q.p,q.id));
        return sqrtl(ans);
    }
};
```

10. 射线网格与轴对齐包围盒 AABB

10.1 射线与三角网格首交点

`rayMesh(r,f,k,p,cull)` 求射线与三角网格最近的非负参数交点。`k` 满足 `p=r.a+k*(r.b-r.a)`。

- `cull=false`：双面网格，适合一般相交查询；
- `cull=true`：只打正面，要求面方向一致；
- 射线与某面共面时该面不产生唯一交点。

复杂度 `O(F)`。大量重复查询可在外层建立 BVH/KD 树；本函数适合作为叶节点求交核心。

```
bool rayMesh(L3 r,const vector<Tri3>& f,ld& k,P3& p,bool cull=false){ // k,p: 最近交点
    bool ok=false; k=INF;
    for(auto t:f){
        ld z; P3 q;
        if(rayTri(r,t,z,q,cull)&&z<k) k=z,p=q,ok=true;
    }
    return ok;
}
```

10.2 AABB 包含、距离与相交

`Box3{lo,hi}` 表示轴对齐闭包围盒，要求每一维 `lo<=hi`。

- `inBox(p,b)`：返回 `0` 外、`1` 边界、`2` 内。
- `disBox(p,b)`：点到闭盒的最短距离。
- `boxCross(a,b)`：两个闭盒是否相交，边界接触算相交。

```
struct Box3 { P3 lo,hi; };
int inBox(P3 p,Box3 b){ // 0 外,1 边界,2 内
    bool bd=false;
    for(int k=0;k<3;k++){
        ld x=coord(p,k),l=coord(b.lo,k),r=coord(b.hi,k);
        if(x<l-EPS||x>r+EPS) return 0;
        if(eq(x,l)||eq(x,r)) bd=true;
    }
    return bd?1:2;
}
ld disBox(P3 p,Box3 b){
    ld s=0;
    for(int k=0;k<3;k++){
        ld x=coord(p,k),l=coord(b.lo,k),r=coord(b.hi,k);
        if(x<l) s+=(l-x)*(l-x); else if(x>r) s+=(x-r)*(x-r);
    }
    return sqrtl(s);
}
bool boxCross(Box3 a,Box3 b){ // 闭盒相交
    for(int k=0;k<3;k++)
if(max(coord(a.lo,k),coord(b.lo,k))>min(coord(a.hi,k),coord(b.hi,k))+EPS) return false;
    return true;
}
```

10.3 射线 / 线段与 AABB：slab 法

`rayBox(r,b,l,h)` 返回射线位于盒内的参数区间 `[l,h]`；相切时 `l==h`。`segBox` 把参数再限制到 `[0,1]`。射线方向要求非零，复杂度 `O(1)`。

```
bool rayBox(L3 r,Box3 b,ld& l,ld& h){ // l,h: 射线在盒内的参数区间
```

```
    P3 d=r.b-r.a; l=0,h=INF;
    for(int k=0;k<3;k++){
        ld x=coord(r.a,k),v=coord(d,k),lo=coord(b.lo,k),hi=coord(b.hi,k);
        if(sgn(v)==0){ if(x<lo-EPS||x>hi+EPS) return false; continue; }
        ld a=(lo-x)/v,c=(hi-x)/v; if(a>c) swap(a,c);
        l=max(l,a),h=min(h,c);
        if(l>h+EPS) return false;
    }
    return h>=-EPS;
}
bool segBox(L3 s,Box3 b,ld& l,ld& h){ // l,h 限制在闭线段参数 [0,1]
    if(!rayBox(s,b,l,h)||l>1+EPS) return false;
    l=max(0.0L,l),h=min(1.0L,h);
    return l<=h+EPS;
}
```

11. 球面几何与经纬度

本节默认球心在原点，经度 `lon`、纬度 `lat` 均使用弧度。经度沿 xy 平面从正 x 轴转向正 y 轴，纬度北半球为正。

11.1 经纬度与空间坐标

`fromLonLat(lon,lat,r)` 把经纬度转为半径 `r` 球面点。`toLonLat(p)` 返回 `{经度,纬度}`，经度范围 `[-PI,PI]`、纬度范围 `[-PI/2,PI/2]`。

```
P3 fromLonLat(ld lon,ld lat,ld r=1){
    ld c=cosl(lat);
    return {r*c*cosl(lon),r*c*sinl(lon),r*sinl(lat)};
}
pair<ld,ld> toLonLat(P3 p){
    p=unit(p);
    return {atan2l(p.y,p.x),asinl(clamp1(p.z))};
}
```

11.2 大圆距离、交点与小弧包含

- `greatCircleDistance(a,b,r)`：方向 `a,b` 在半径 `r` 球面上的最短大圆弧长。
- `greatCircleCross(a,b,c,d)`：大圆 `ab` 与大圆 `cd` 的两个对径交点；大圆重合时返回空。
- `onMinorArc(p,a,b)`：单位方向 `p` 是否位于 `a` 到 `b` 的较短闭大圆弧上。端点互为对径时短弧不唯一，不属于接口前提。

```
ld greatCircleDistance(P3 a,P3 b,ld r=1){ return r*angleVec(a,b); }
vector<P3> greatCircleCross(P3 a,P3 b,P3 c,P3 d){ // 参数为四个非共线球面方向
    P3 x=cross(a,b),y=cross(c,d),p=cross(x,y);
    if(sgn(len(p))==0) return {};
    p=unit(p);
    return {p,-p};
}
bool onMinorArc(P3 p,P3 a,P3 b){ // p,a,b: 非零方向，a 与 b 不对径
    ld x=angleVec(a,p)+angleVec(p,b),y=angleVec(a,b);
    return fabsl(x-y)<=EPS;
}
```

11.3 球面三角形面积

`sphereTriArea(a,b,c,r)` 返回三个球面方向在半径 `r` 球上围成的较小有向选择对应的非负面积。先归一化到单位球，再用稳定的立体角公式；复杂度 `O(1)`。

```
ld sphereTriArea(P3 a,P3 b,P3 c,ld r=1){ // a,b,c: 非零球面方向
    a=unit(a),b=unit(b),c=unit(c);
    ld x=fabsl(2*atan2l(triple(a,b,c),1+dot(a,b)+dot(b,c)+dot(c,a)));
    return x*r*r;
}
```

---

12. 常用公式与结论

12.1 向量与点线面

- `a dot b=|a||b|cos(theta)`；垂直等价于点积为零。
- `|a cross b|=|a||b|sin(theta)`；平行等价于叉积为零。
- `a dot (b cross c)` 是有向平行六面体体积；绝对值除以 `6` 是对应四面体体积。
- 点 `p` 到直线 `ab` 距离：`|(p-a) cross (b-a)|/|b-a|`。
- 点 `p` 到平面 `n dot (x-a)=0` 距离：`|n dot (p-a)|/|n|`。
- 两异面直线方向为 `u,v` 时，距离为 `|(p-q) dot (u cross v)|/|u cross v|`。
- 三个向量共面等价于标量三重积为零；四点共面等价于 `orient(a,b,c,d)=0`。

12.2 球、圆锥与柱体

- 球表面积 `4PI*r^2`，球体积 `4PI*r^3/3`。
- 球冠高 `h`：面积 `2PI*r*h`，体积 `PI*h^2*(3r-h)/3`。
- 圆柱体积 `PI*r^2*h`，侧面积 `2PI*r*h`。
- 圆锥体积 `PI*r^2*h/3`，侧面积 `PI*r*l`，其中 `l` 为母线长。
- 棱锥体积 `S*h/3`。
- 棱台/圆台体积 `h*(S1+S2+sqrt(S1*S2))/3`。

- 正四面体边长 `a`：表面积 `sqrt(3)*a^2`，体积 `sqrt(2)*a^3/12`，外接球半径 `sqrt(6)*a/4`，内切球半径 `sqrt(6)*a/12`。
- 球带高 `h` 的面积也是 `2PI*r*h`，与球带所在纬度无关。
- 球面两点的弦长为 `2r*sin(theta/2)`，最短大圆弧长为 `r*theta`。
- 单位球面三角形面积等于其球面角超 `A+B+C-PI`；半径为 `r` 时再乘 `r^2`。

12.3 多面体与拓扑

- 闭合定向三角网格体积可由 `sum(a dot (b cross c))/6` 求得，不要求原点在内部。
- 凸多面体的线性函数最大值必在顶点取得。
- 简单凸多面体满足 Euler 公式 `V-E+F=2`；三角剖分凸多面体一般位置下 `E=3V-6,F=2V-4`。
- 平面截球得到圆；圆心是球心在平面上的投影，半径平方为 `r^2-d^2`。
- 四个不共面的点唯一确定球；三个不共线点在其平面内唯一确定圆。

12.4 常见错误

- 把三维叉积当标量：三维 `cross` 返回向量，面积是其长度，定向通常再与另一个向量点乘。
- 用二维跨立实验判断空间线段相交：两线段可能异面，应使用 `segClosest` 的距离。
- 点到平面投影使用绝对距离：投影必须保留 `side` 的符号，本板子公式已处理。
- 把两直线最近点公式用于平行线：分母为零，必须单独投影。
- 三角网格面方向混乱：有向体积会互相抵消，凸多面体包含判定也会反向。
- 用 `EPS` 写 `sort` 比较器：会破坏严格弱序；原始点排序使用精确字典序。
- 忘记退化输入：`unit`、平面、三角形、无限直线相关函数均要求对应向量非零。
- `convexHull3` 的输入若全共面，应先投影到二维求凸包，不能把空面集理解为空点集。
- 空间圆不仅要满足到圆心距离为 `r`，还必须位于圆所在平面。
- 经纬度题中角度通常以度输入，调用球面接口前需乘 `PI/180` 转为弧度。
- AABB slab 法中的平行轴不能做除法；应检查射线起点该坐标是否落在区间内。

---

13. 最小复制组合与专题边界

题型	复制章节
点线面关系、距离、交点	`0 + 1 + 2 + 3` 对应小节
绕轴旋转	`0 + 1 + 4.1`
空间三角形相交 / 距离	`0 + 1 + 2.4 + 3.1 + 5.1~5.4`
共面空间多边形	`0 + 1 + 3.1 + 4.2 + 5.5`
球与线、面、球相交	`0 + 1 + 2.1 + 3.1 + 6` 对应小节

题型	复制章节
空间圆交点 / 最近点	`0 + 1 + 2/3` 对应小节 `+ 4.2 + 6.2 + 6.6`
最小覆盖球	`0 + 1 + 6.5 + 6.7`
三角网格面积、体积、重心	`0 + 1 + 5.1 + 7.1/7.2`
三维凸包	`0 + 1 + 5.1 + 7.2 + 8`
点在凸多面体内	`0 + 1 + 5.1 + 7.3`
三维最近点对 / 最近邻	`0 + 1 + 9`
射线网格 / AABB	`0 + 1 + 5.3 + 10`
经纬度与球面几何	`0 + 1 + 3.5 + 11`

全文代码块按顺序拼接可以共同编译。实际提交时只复制依赖链上的章节。

以下属于独立大型专题，没有混入本通用板子：三维 Delaunay / Voronoi、半空间交构造多面体、多面体布尔运算、动态碰撞检测、旋转四元数与刚体矩阵库。它们退化约定和代码规模较大，遇到对应题目应使用专项板子。